

Unidade Lógica e Aritmética

Autores: Ericka Almeida de Lima, Gustavo Leão, Felipe Queiroz

Universidade Estadual de Feira de Santana - Av.
Transnordestina, Novo Horizonte, CEP 44036-900 -
Feira de Santana-Bahia

erickauefs@gmail.com

barreferupu@gmail.com

gustavoleao343@gmail.com

Resumo. *O presente relatório descreve o processo de desenvolvimento do protótipo da Unidade Lógica e Aritmética. Utilizando a linguagem de descrição de hardware Verilog tipo estrutural e aplicando entrada de dois operandos de 4 bits, operações aritméticas soma, subtração, multiplicação, divisão e lógicas AND OR E XOR selecionáveis por meio de chaves da placa DE10-Lite. Desenvolvido no ambiente Quartus II, a metodologia utilizada baseou-se na criação de circuitos combinacionais de operações aritméticas e lógicas, aparato de seleção de operação através de um multiplexador e exibição dos resultados obtidos em dois dos displays de 7 segmentos presentes na placa. Essa configuração permitiu a implementação e visualização prática da lógica combinacional aplicada aos circuitos digitais.*

1. Introdução

Etimologicamente, o termo “digital” tem sua origem no latim *digitalis*, que significa “relacionado aos dedos”, característica do sistema indo-arábico de contagem que deu origem ao sistema numérico decimal utilizado atualmente na maioria dos países[1]. De forma semelhante, o desenvolvimento de circuitos digitais modernos baseia-se na manipulação de informações por meio de um sistema binário. Nesse contexto, a informação é representada e transmitida através de sinais elétricos que assumem apenas dois níveis lógicos distintos: alto (1) e baixo (0), com a característica de ser posicional [2].

Para os circuitos digitais combinacionais, as suas saídas dependem exclusivamente da informação recebida em suas entradas. Esse processamento é feito através da associação de portas e/ou blocos lógicos que realizam operações aritméticas e booleanas com os sinais de entrada.

Com base nisso, desenvolveu-se um protótipo de uma Unidade Lógica e Aritmética (ULA) de duas variáveis com 4 bits cada. A ULA caracteriza o núcleo básico de processadores presentes em todos os circuitos digitais, como a calculadora moderna. O projeto consiste no desenvolvimento de um circuito combinacional que recebe dois

operandos de 4 bits e entrada *Carry in*, ativa para soma, e realiza diversas operações selecionáveis, utilizando a linguagem de descrição de hardware Verilog para a prototipação em uma placa de desenvolvimento FPGA DE10-Lite.

O sistema implementado contempla operações aritméticas de soma ($A+B$), subtração ($A-B$), multiplicação ($A \times B$) e divisão (A/B), além de operações lógicas *AND*, *OR*, e *XOR*. Os resultados das operações, bem como os *flags* de Zero, *Carry Out*, *Borrow Out* e Erro, foram exibidos em LEDs e em dois displays de sete segmentos para facilitar a interpretação. Para a seleção das operações possíveis, determinou-se entradas únicas (exceto para soma) que combinam 3 bits de entrada para 1 saída. Todo o processo foi realizado no ambiente de desenvolvimento (IDE) Quartus II com linguagem de descrição de *hardware* Verilog tipo estrutural.

A disposição do relatório abrange: Introdução, Fundamentação Teórica, Metodologia, Resultados e Discussões, e Conclusão. Na Fundamentação Teórica são descritos os conceitos essenciais para o desenvolvimento do protótipo da ULA. Na Metodologia são apresentados as estratégias de organização e sistematização do projeto, a minimização do circuito e a ordem de codificação em Verilog estrutural. Na seção de Resultados e Discussões são expostos os testes, simulações e a demonstração do funcionamento do protótipo na plataforma DE10-Lite na subseção Manual de Uso, juntamente com a análise dos resultados obtidos e dificuldades de projeto. Ao final, na Conclusão, são destacados os objetivos cumpridos e as possíveis implementações futuras para ampliar os resultados do produto final.

2. Fundamentação Teórica

Para o pleno entendimento da resolução apresentada para o problema, é de suma importância a compreensão dos conceitos teóricos os quais fundamentam todo o desenvolvimento do projeto. Este tópico descreve os fundamentos teóricos dos circuitos construídos, da placa utilizada e conceitos relacionados.

2.1. FPGA e Placa DE10-Lite

Um FPGA (Field-Programmable Gate Array) é um circuito integrado reconfigurável, o que o diferencia de chips com lógica fixa, como microcontroladores e ASICs [3]. O núcleo de um FPGA é uma matriz de blocos lógicos configuráveis (CLBs), que podem ser programados para executar funções lógicas e se interconectam por uma rede de comutação programável [4]. Essa arquitetura possibilita um processamento massivo e paralelo, uma vantagem em relação aos processadores que operam sequencialmente [4]. A programação é feita com Linguagens de Descrição de Hardware (HDLs) como o Verilog (linguagem utilizada no desenvolvimento do projeto) e VHDL, que descrevem o circuito em vez de um software. Um software de síntese, como o Quartus II, utilizado na resolução do problema, traduz o código em uma configuração de bits que define o circuito interno do FPGA [5].

As placas de desenvolvimento, como a placa DE10-Lite utilizada para o desenvolvimento do projeto, integram um FPGA com periféricos e interfaces externas para facilitar a prototipagem, eliminando a necessidade de projetar circuitos complexos

[6]. As funcionalidades da DE10-Lite, como botões, chaves, LEDs e conectores GPIO, são controladas através do FPGA, o que permite testar lógicas sem componentes externos. O FPGA central, o Altera MAX 10, é uma escolha estratégica por ser uma solução de chip único com baixo consumo de energia e menor custo, ideal para aplicações de controle e processamento de dados, além disso, A placa possui um programador USB Blaster embutido, facilitando o processo de carregamento de designs para o FPGA. Essa característica torna o kit auto-suficiente e dispensa a necessidade de hardware de programação externo [7][8].

2.2. Circuitos aritméticos

Os circuitos aritméticos representam um componente fundamental na arquitetura do projeto, sendo seu funcionamento essencial para a eficácia do produto final. Esta seção tem por objetivo descrever a estrutura e a lógica de operação dos circuitos aritméticos empregados, detalhando sua construção e princípio de funcionamento.

2.2.1 Circuito Somador

A soma de números binários foi realizada por circuitos somadores (ou *adders*). O processo de soma é análogo ao da aritmética decimal, começando pelo bit menos significativo e propagando o "vai um" (*carry*) para a próxima coluna.

Meio Somador (*Half Adder*): É o circuito mais simples, que soma apenas dois bits de entrada (A e B). Ele gera duas saídas: a Soma (S) e o Vai Um (Cout). O Meio Somador é essencialmente uma porta XOR para a soma e uma porta AND para o *carry* [9]. Este circuito é chamado de "meio" porque não considera um "vai um" de entrada da coluna anterior [10]. Dentro do módulo do somador de 4 bits do projeto, o meio somador foi desconsiderado, pois o *carry in* inicial é requisito do problema, no entanto, o meio somador teve suma importância no módulo do Multiplicador.

Para somar números com mais de um bit, foi necessário um circuito que também levava em conta o "vai um" vindo da coluna anterior (*Cin*). O Somador Completo possui três entradas (A, B e *Cin*) e duas saídas (Soma e *Cout*), sendo ideal para operações em cascata [4]. Dessa forma, o módulo constituído de um somador completo de 1 bit, foi repetido 4 vezes, ligando os respectivos *carry outs* nos respectivos *carry ins* das seguintes operações, gerando uma soma completa de 4 bits e *Carry out*. Esse processo é representado na figura 1.

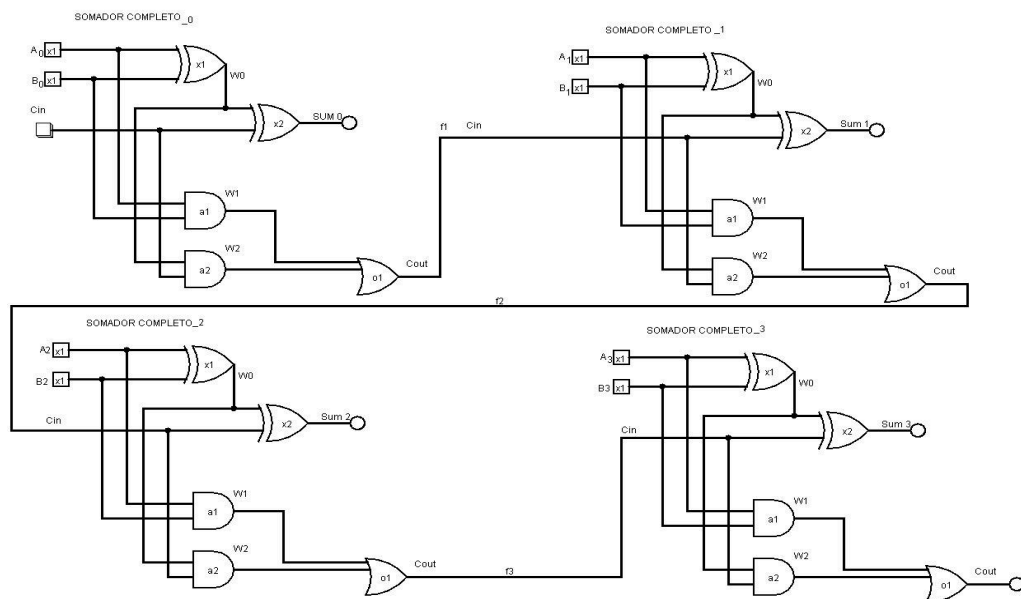


Figura 1. Representação do somador completo de 4 bits no logisim.Fonte: Domínio dos Autores.

2.2.2 Circuito Subtrator

Para a subtração de números binários, a lógica básica do somador permanece, mas com a introdução do conceito de "empréstimo" (*borrow*).

Meio Subtrator (*Half Subtractor*): Este circuito realiza a subtração de dois bits de entrada (A e B). As saídas são a Diferença (D) e o Empréstimo (Bout). Assim como no Meio Somador, ele não leva em conta um empréstimo vindo da coluna anterior [10]. O circuito pode ser implementado com portas lógicas, onde a Diferença é uma porta XOR e o Empréstimo é uma porta AND entre a negação do primeiro bit e o segundo.

Para o Subtrator Completo (*Full Subtractor*), usou-se subtrações com múltiplos bits, que levam em conta o "empréstimo" da coluna anterior (*Bin*). Ele possui três entradas (A, B e *Bin*) e duas saídas (Diferença e *Bout*), podendo ser combinado em cascata para subtrações de vários bits [10]. Esse processo é representado na figura 2.

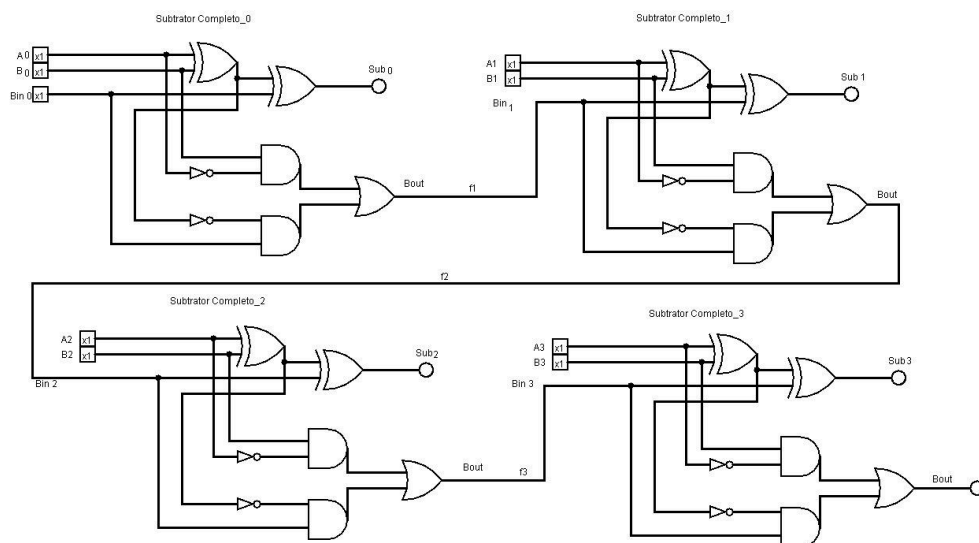


Figura 2. Representação do subtrator completo de 4 bits. Fonte: Domínio dos Autores.

2.2.3. Circuito Multiplicador

A implementação do circuito multiplicador de 4 bits foi realizada utilizando a lógica combinacional, fundamentada na conexão em cascata de meio-somadores (*Half Adders*) e somadores completos (*Full-Adders*). O algoritmo de funcionamento simula a multiplicação binária manual.

Inicialmente, são gerados os produtos parciais. Cada bit do multiplicando é combinado com cada bit do multiplicador por meio de portas lógicas *AND*. Para uma multiplicação de dois números de 4 bits ($A_3A_2A_1A_0$ e $B_3B_2A_1B_0$), isso resulta em 16 produtos parciais ($A_0B_0, A_1B_0, A_2B_0, A_3B_0, \dots, A_3B_3$).

Em seguida, esses produtos parciais são somados de forma enfileirada, utilizando uma cascata de somadores. A primeira linha de soma é realizada, e o *Carry Out* de cada soma é propagado como *Carry In* para a soma da coluna adjacente de maior significância. O resultado final é um produto de 8 bits ($P_7P_6P_5P_4P_3P_2P_1P_0$) sendo o bit menos significativo (P_0) o resultado da primeira porta *AND* (A_0B_0). Os bits subsequentes são o resultado das somas realizadas pelos meio-somadores e somadores completos, e o bit mais significativo (P_7) é o último *Carry Out* gerado pela cascata de somadores.

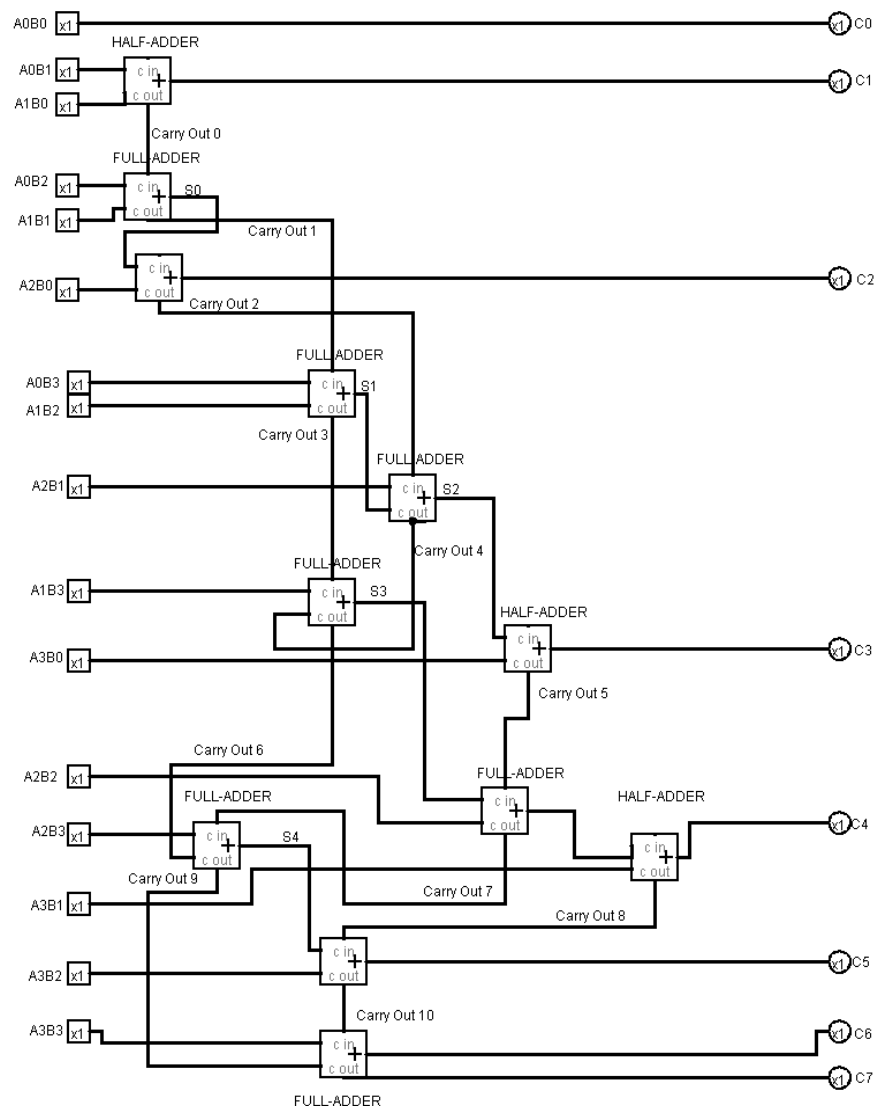


Figura 3 - Circuito Multiplicador com 1 saída de 8 bits. Fonte: Domínio dos Autores.

2.2.4. Circuito Multiplexador

Para atender à necessidade de um circuito cuja saída pudesse ser alterada dinamicamente conforme o tipo de entrada, implementou-se um circuito multiplexador.

O multiplexador, também conhecido como seletor de dados, é um circuito lógico combinacional projetado para selecionar uma entre várias linhas de entrada e direcioná-la para uma única linha de saída. A seleção da entrada que será transmitida para a saída depende do conjunto de entradas de controle, denominadas chaves seletoras ou endereço [4].

Para duas entradas de dados e uma chave seletora, o multiplexador é composto por duas portas lógicas *AND*, uma porta *NOT* e uma porta *OR*, conforme ilustra a figura

?. O primeiro *AND* possui entrada I_1 e chave seletora S . O segundo *AND* possui entrada I_0 e chave seletora negada pela porta lógica *NOT*. As duas saídas geradas pela operação são somadas pela porta *OR*, gerando a saída Z do multiplexador.

A saída Z será igual à entrada I_1 caso a entrada de seleção seja 1, pois o segundo *AND* terá saída zero causada pela negação da entrada ativa da seleção. Já para saída Z igual à entrada I_0 , a chave seletora deve permanecer em estado lógico zero, que negado, resulta em saída ativa 1 para o segundo *AND*.

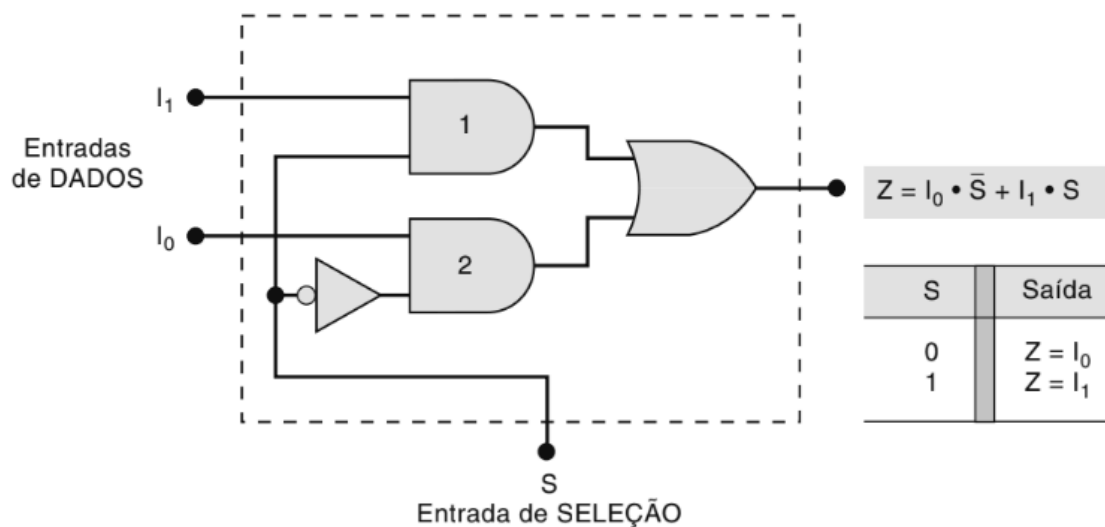


Figura 4 - Circuito Lógico do Multiplexador com 2 entradas de dados e 1 de seleção.Fonte:[15].

Para a porta lógica *AND* 1 habilitada, a seleção deve ser 1—A expressão booleana é:

$$Z = I_0 \cdot 0 + I_1 \cdot 1 = I_1$$

Para a porta lógica *AND* 2 habilitada, a seleção deve ser 0—A expressão booleana é:

$$Z = I_0 \cdot 1 + I_1 \cdot 0 = I_0$$

Conforme a quantidade de entradas de dados no multiplexador, será necessário aumentar a quantidade de entradas seletoras. Se um multiplexador tem N entradas de dados, ele precisará de m chaves seletoras, onde N é igual a 2 elevado à potência de m . Para saber a quantidade de chaves necessárias, deve-se utilizar a relação inversa:

$$m = \log_2(N)$$

Portanto, para 8 entradas no multiplexador ($N=8$), serão necessárias 3 chaves seletoras no multiplexador (Figura 5). Todas as 3 chaves são ligadas à cada entrada e conforme a operação de seleção (Entre 000 e 111), e as entradas selecionáveis são negadas.

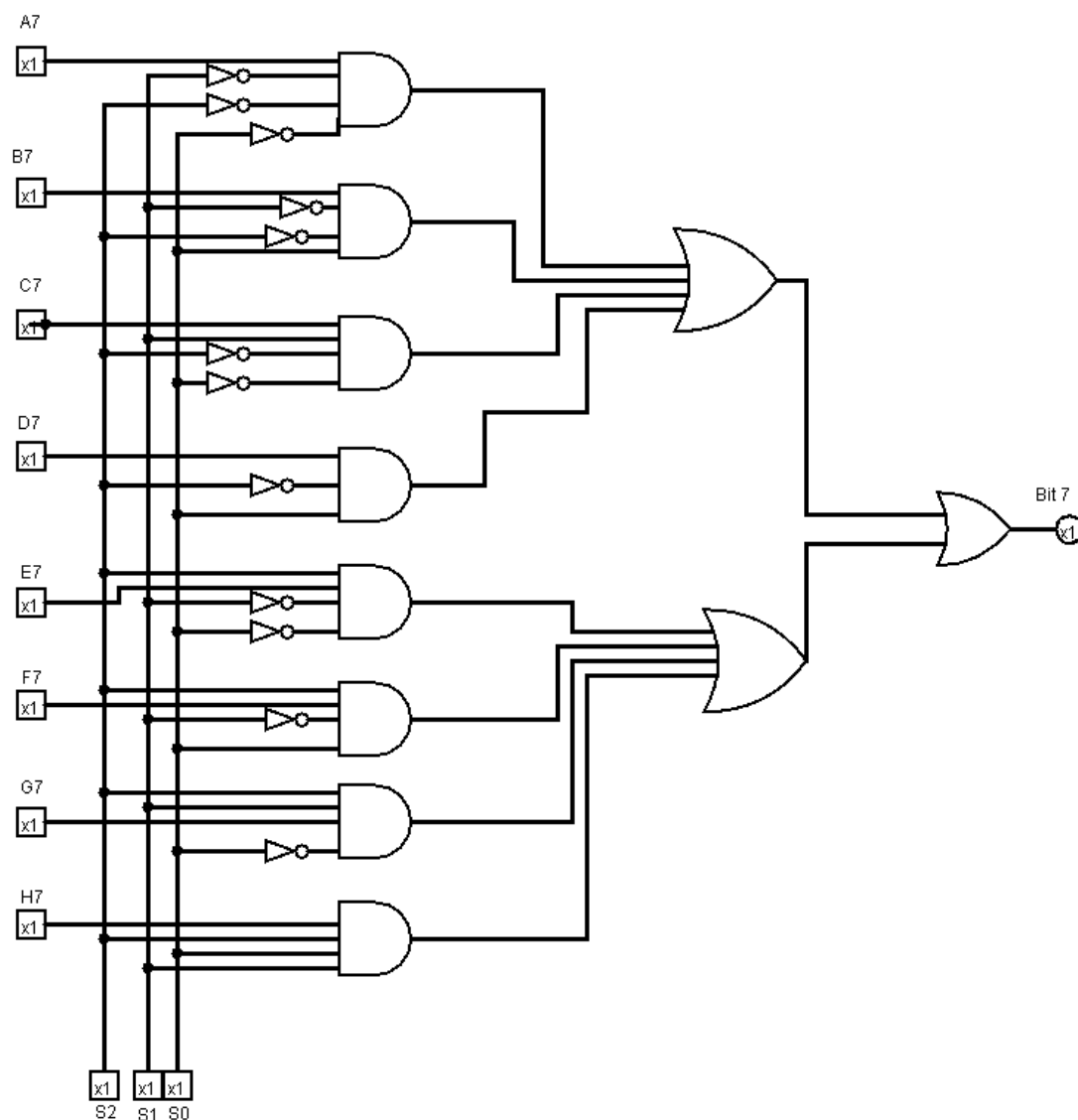


Figura 5 - Multiplexador com 8 entradas de dados e 3 de seleção. Fonte: Domínio dos Autores.

2.2.4. Circuito Divisor

O processo de divisão binária implementado pelo circuito é análogo ao método de divisão decimal, baseando-se em um algoritmo de subtrações sucessivas. Seu funcionamento inicia-se a partir do bit mais significativo (MSB) do dividendo. Para cada bit do quociente a ser determinado, uma subtração de teste é realizada entre o dividendo parcial e o divisor.

A decisão sobre o valor do bit do quociente (0 ou 1) depende exclusivamente do resultado da subtração, que é indicado pelo valor de *Borrow Out* como sinal da operação-*Borrow Out* 0 = Subtração com resultado positivo, *Borrow Out* = 1, Subtração com resultado negativo.

Se a subtração não gera um *Borrow out* ($Bout = 0$), significa que o divisor é menor ou igual ao dividendo parcial. Neste caso, o bit correspondente do quociente é definido como 1 (*Borrow Out* negado). O resultado da subtração é mantido como o novo dividendo parcial para a próxima etapa de comparações.

Se a subtração gera um *Borrow out* ($Bout = 1$), indica que o divisor é maior que o dividendo parcial. O bit do quociente é, então, definido como 0 (*Borrow Out* negado). O resultado da subtração é descartado, e o valor do dividendo parcial original é restaurado para ser utilizado na etapa seguinte.

Este ciclo combinacional se repete a cada bit do dividendo, deslocando e incorporando o próximo bit para formar o novo dividendo parcial em cada etapa, até que todos os bits tenham sido processados.

Para implementar o algoritmo de divisão, o circuito foi estruturado em estágios, onde cada estágio é responsável por calcular um bit do quociente, conforme ilustra a figura ?. Os componentes centrais são meio subtratores e subtratores completos, multiplexadores e portas lógicas *NOT*.

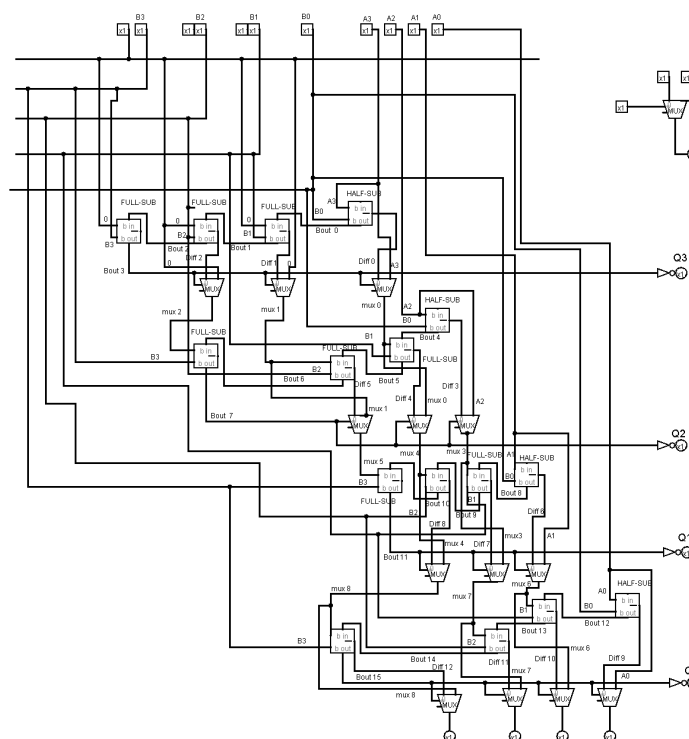


Figura 6. Circuito Divisor de 4 bits. Fonte: Domínio dos autores.

A decisão de manter o resultado da subtração ou restaurar o valor anterior é implementada por um conjunto de multiplexadores 2x1 e a chave seletora é o resultado direto do Bout de cada subtração realizada. Quando Bout = 0, os multiplexadores selecionam a saída da unidade de subtração (a diferença), repassando-a como o novo dividendo parcial. Para Bout = 1, os multiplexadores selecionam a entrada original (o dividendo parcial anterior considerando os zeros acrescidos), descartando o resultado da subtração e restaurando o valor.

Ao final de todos os agrupamentos possíveis com cada bit do dividendo, são gerados restos que correspondem às saídas dos multiplexadores finais.

2.3. Circuitos lógicos

Os circuitos lógicos também apresentam suma importância como parte dos requisitos do problema. Esta seção tem por objetivo descrever a estrutura e a lógica de operação dos circuitos lógicos empregados, detalhando sua construção e princípio de funcionamento.

2.3.1 Operações lógicas: AND, OR e XOR

A maneira em que as operações lógicas foram implementadas no protótipo desenvolvido se baseia na seguinte lógica: todas as respectivas entradas A, se unem em uma porta lógica (AND, OR ou XOR) junto a todas as respectivas entradas B, formando os pares (A0, B0), (A1, B1), (A2, B2), (A3, B3). Essa formação de pares foi repetida em diferentes módulos para cada porta lógica, gerando os circuitos que podem ser observados nas figuras 6, 7 e 8.

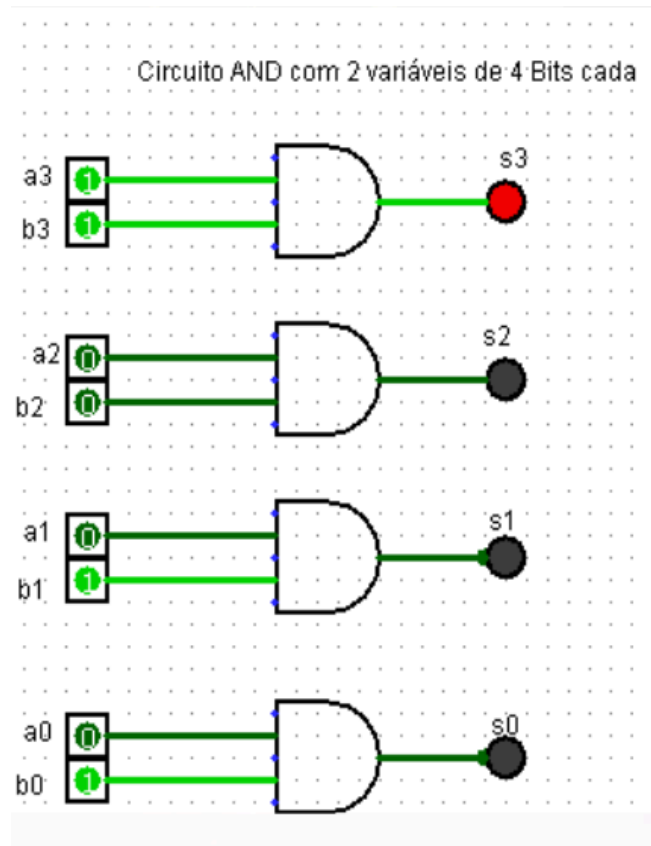


Figura 7 - Circuito AND. Fonte: Domínio dos Autores.

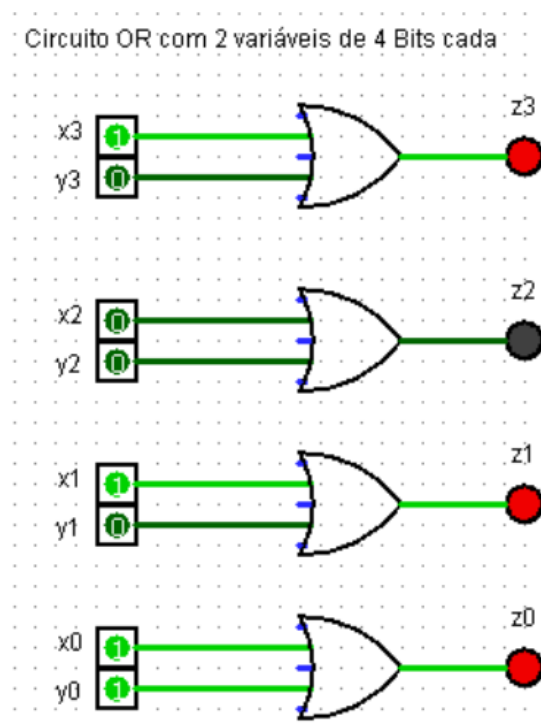


Figura 8 - Circuito OR. Fonte: Domínio dos Autores.

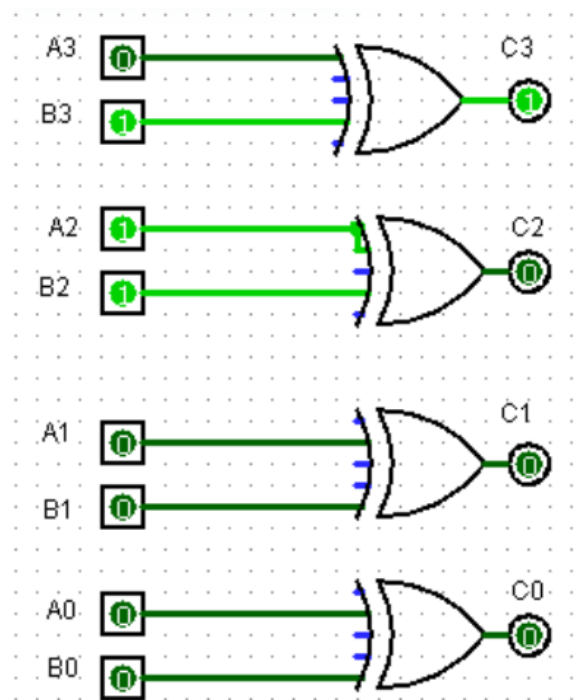


Figura 9 - Circuito XOR. Fonte: Domínio dos Autores.

3. Metodologia

Para o desenvolvimento dos circuitos, foram empregadas metodologias formais que garantem a correta especificação, otimização e implementação do comportamento lógico desejado. A presente seção tem como objetivo, portanto, expor e detalhar cada uma dessas técnicas.

3.1. Tabela verdade

A utilização do método de representação e análise de proposições lógicas, conhecido como Tabela Verdade, foi essencial para o desenvolvimento do projeto, possibilitando a compreensão do funcionamento dos circuitos construídos. Para a realização da análise, é necessário declarar todas as variáveis de entrada que compõem o problema a ser resolvido, de modo a identificar suas respectivas saídas com valores lógicos verdadeiros. A partir desse processo, obtêm-se as expressões lógicas equivalentes à descrição da tabela.

Tabela 1. Tabela verdade do somador completo.

A	B	Carry In	Soma	Carry Out
0	0	0	0	0
0	0	1	1	0

0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Tabela 2. Tabela verdade do subtrator completo.

A	B	Borrow In	Diferença	Borrow Out
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Tabela 3. Tabela verdade da multiplicação em binário.

A	B	A.B
0	0	0
0	1	0
1	0	0
1	1	1

O produto final apresentado no projeto corresponde à divisão realizada entre os 4 bits de A e B. A tabela a seguir é apenas um exemplo, ilustrando o mesmo processo que ocorre quando se utilizam apenas dois bits de A e B, mostrando de forma simplificada a lógica da operação.

Tabela 4. Tabela verdade da divisão entre operandos de 2 bits.

A[1]	A[0]	B[1]	B[0]	Q[1]	Q[0]
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	0	0
0	0	1	1	0	0
0	1	0	0	ERRO	ERRO
0	1	0	1	0	1
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	ERRO	ERRO
1	0	0	1	1	0
1	0	1	0	0	1
1	0	1	1	0	0
1	1	0	0	ERRO	ERRO
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	1	0	1

Para realizar a divisão utilizando os 4 bits de A e B, foi necessário implementar um multiplexador que verificasse se o valor de A era menor que o valor de B, de modo a conservar o resultado para a próxima subtração entre A e B. Esse multiplexador recebe como entradas o sinal de A, a diferença entre A e B e a chave de seleção Borrow Out, que indica se a subtração resultou em valor negativo, ou seja, sinaliza se A é menor que B.

Tabela 5. Tabela verdade do multiplexador da divisão.

D	A	Borrow Out	Saída
0	0	0	0

0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

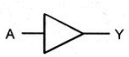
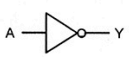
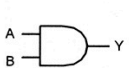
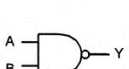
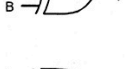
Logic function	Logic symbol	Truth table	Boolean expression															
Buffer		<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td></tr></table>	A	Y	0	0	1	1	$Y = A$									
A	Y																	
0	0																	
1	1																	
Inverter (NOT gate)		<table><tr><th>A</th><th>Y</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	A	Y	0	1	1	0	$Y = \bar{A}$									
A	Y																	
0	1																	
1	0																	
2-input AND gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	0	1	0	0	1	1	1	$Y = A \cdot B$
A	B	Y																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
2-input NAND gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	1	1	0	1	1	1	0	$Y = \overline{A \cdot B}$
A	B	Y																
0	0	1																
0	1	1																
1	0	1																
1	1	0																
2-input OR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	1	$Y = A + B$
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
2-input NOR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	0	$Y = \overline{A + B}$
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
2-input EX-OR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	A	B	Y	0	0	0	0	1	1	1	0	1	1	1	0	$Y = A \oplus B$
A	B	Y																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
2-input EX-NOR gate		<table><tr><th>A</th><th>B</th><th>Y</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	A	B	Y	0	0	1	0	1	0	1	0	0	1	1	1	$Y = \overline{A \oplus B}$
A	B	Y																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

Figura 10. A Tabela verdade dos elementos lógicos. Fonte[11].

3.2. Mapa de Karnaugh simplificado pelo método da Soma de Produtos (SOP)

Por conseguinte, à medida que as variáveis de entradas dos circuitos aumentavam, dificultava a simplificação das expressões booleanas pelo método convencional. Com base nesse cenário, tornou-se essencial a aplicação do método gráfico Mapa de Karnaugh para simplificar as expressões dos circuitos e diminuir o número de portas lógicas do projeto, a fim de, achar a solução mais barata possível para o funcionamento utilizando o método Soma de Produtos(SOP). Sob esse viés, a Tabela Verdade é construída e de

acordo com as combinações das variáveis dispostas na grade do mapa, são transferidos os 0s e 1s obtidos nas saídas da etapa anterior, após esse processo, deve identificar e agrupar os 1s nos maiores grupos de potências de 2 possíveis, em sequência, cada grupo irá originar uma expressão formada pelos produtos, ou seja, as portas AND, com a combinação das expressões formadas pelos grupos em uma porta OR, é feito a Soma dos Produtos. Deste modo, foi viável a implementação do circuito mais econômico na solução do problema.

3.3. Display de sete segmentos ânodo comum

Como requisito de projeto, os resultados das interações com os usuários devem ser exibidos em LEDs e em dois display de 7 segmentos. Dessa maneira, as *Flags* e os resultados de 0 a 99 devem ser corretamente representados pelos componentes, e para isso, se tornou imprescindível consultar o manual da placa DE10-Lite [8], uma vez que, ele explicava os comportamentos característicos dos elementos da placa, como, por exemplo, os Display de Ânodo Comum, que para acender um segmento é preciso aplicar nível lógico 0, ou seja, o seu funcionamento é com lógica invertida, conforme a figura 2 ilustra.

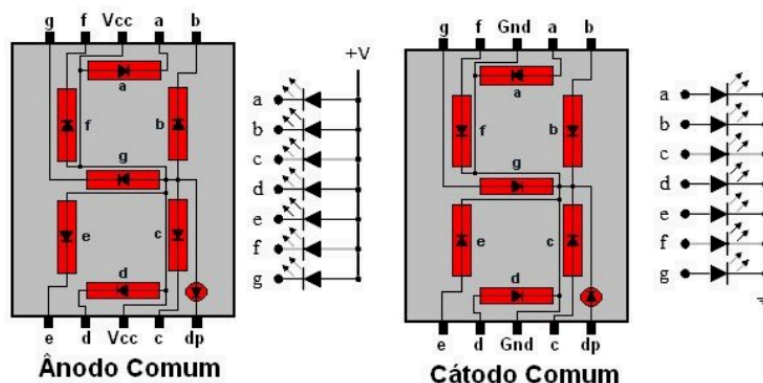


Figura 11. Displays de 7 segmentos nos modelos de ânodo comum e cátodo comum. (Fonte: Autor desconhecido).

3.4. Expressões simplificadas pelo método SOP por segmento

Em virtude disso, consideramos os dois Display de maneira individual, o primeiro irá representar as unidades, enquanto, o segundo às dezenas. Sobre essa perspectiva, os 14 segmentos foram nomeados de “a” a “n”, foram consideradas as possibilidades de estado ativo e não ativo para números entre 0 e 99. Nessa óptica, a tabela verdade foi construída com base nesses princípios de operação para representar os números. A figura 12 informa a disposição das letras sobre os segmentos dos Displays.



Figura 12. Display de 14 segmentos, identificados de “a” a “n”. Fonte: Domínio dos autores.

Conforme mencionado anteriormente, a aplicação do Mapa de Karnaugh possibilitou a simplificação das expressões booleanas pelo método Soma de Produtos (SOP), assim, para cada segmento dos dois displays foram necessárias uma Tabela Verdade. Cada tabela verdade produziu uma saída com a expressão booleana mais simplificada e responsável por identificar quais segmentos devem acender de acordo com a representação do resultado em decimal.

3.5. Modularização

Como forma de organizar a estruturação e facilitar a manutenção do projeto, utilizou-se o conceito de Modularização [14], decompondo o sistema em blocos funcionais com responsabilidades definidas para cada operação.

Como blocos aritméticos fundamentais, criou-se “MeioSomador”, “Somador”, “MeioSubtrator” e “multiplexador”. O MeioSomador realiza a soma de dois bits (A, B), gerando uma saída de soma (S) e uma saída de transporte (*carry-out*) (Co). É implementado com uma porta *XOR* para a soma e uma porta *AND* para o *carry*.

O módulo Somador realiza a soma de três bits (A, B e um *carry-in* Ci), produzindo uma saída de soma (S) e um *carry-out* (Co). Para o MeioSubtrator, é feita a operação entre dois bits (A, B), gerando a diferença (D) e um empréstimo (*borrow-out*) (Bo). Já o módulo “multiplexador” possui 2 entradas (D, A) de 1 bit cada e 1 chave de seleção(S), direcionando o valor escolhido para a saída R.

Para adicionar as duas entradas de 4 bits necessárias, foram implementados módulos que mapeam suas entradas e saídas conforme os módulos aritméticos fundamentais. O “somador4bits” recebe duas entradas de 4 bits (A, B) e um *carry-in* (CIN). O cálculo é realizado em cascata, instanciando quatro módulos Somador de 1 bit, onde o *carry-out* de um estágio serve como *carry-in* para o próximo. A saída SUM possui 8 bits para acomodar o resultado (somente 5 bits menos significativos são resultado da operação de fato) e o *carry-out* final.

O “subtrator4bits” realiza a subtração de dois números de 4 bits (A, B) com um *borrow-in* (Bin). Sua arquitetura é similar à do somador, instanciando quatro subtratores completos de 1 bit em cascata.

Para o “and_or_4bits” são executadas operações lógicas bit a bit. Para duas entradas de 4 bits (a, b), são gerados 8 bits como saída “s” contendo o resultado da operação *AND* e uma saída “z” com o resultado da operação *OR*. Internamente, instancia portas and_1bit e or_1bit para cada par de bits. O módulo “xor_4bits” define operação básica *XOR* bit a bit entre duas entradas de 4 bits (A, B), produzindo uma saída de 8 bits “S”.

O módulo “divisor” tem como entrada de 4 bits, o dividendo A e o divisor B. O resultado da operação é fornecido para duas saídas: Um quociente (Q) de 4 bits e para a *flag* (fdiv) que sinaliza a ocorrência de uma divisão por zero. Nesse bloco, o módulo multiplexador é instanciado juntamente com blocos de meio subtrator e subtrator completo.

O gerenciamento do fluxo das operações e comandos foram definidos em 3 módulos: “Mux”, “main” e “main2”.

O módulo Mux implementa um multiplexador de 8 entradas para um único bit. Possui sete entradas de dados (S, Sub, M, D, A, O, X) e três chaves de seleção (S2, S1, S0). A saída R reflete a entrada selecionada pelas chaves. Os códigos de seleção 000 e 001 direcionam a mesma entrada (S) da operação soma para a saída única R. Esse módulo é chamado 8 vezes pelo módulo “main” que define 8 entradas de 8 bits cada, que são os resultados de todas as operações aritméticas e lógicas realizadas. As chaves de seleção de 3 bits (Chaves) são compartilhadas entre todas as instâncias, garantindo que a mesma operação seja selecionada para todos os bits, resultando em uma saída de 8 bits (Saida).

O módulo “main2” é o módulo principal e realiza a coordenação das entradas recebidas e integração dos módulos. Ele coordena as operações aritméticas e lógicas e recebe duas entradas de 4 bits (A, B), uma chave de seleção de 3 bits (Selecao) e um *carry-in* (Cin).

Ele instancia todos os módulos de operação de 4 bits (somador4bits, subtrator4bits, Multiplicador, divisor, and_or_4bits, xor_4bits). As saídas de todas essas operações são conectadas às entradas do módulo main. A entrada “Selecao” é conectada às chaves de seleção do main, que escolhe qual resultado de operação será direcionado para a saída Resu.

A partir do resultado final Resu, são geradas diversas *flags* de estado, como fzero (zero), fdiv (divisão por zero), flag_bout (*borrow-out*), flag_cout (*carry-out*) e flag_over (*overflow*). O resultado de 8 bits é enviado a um módulo “display” que define todas as expressões booleanas para cada segmento para visualização em dois displays. Para este módulo, definiu-se 14 segmentos necessários para a representação decimal de 0 a 99,

4. Resultados e Discussões

Em concordância com o problema, foi projetado as Unidades Lógicas e Aritméticas (ULAS), que são fundamentais para a realização das operações matemáticas e lógicas em processadores na placa DE10-Lite de maneira funcional, tais como, soma, subtração, multiplicação e divisão, além disso, as operações lógicas AND, OR, e XOR entre os 4 bits de dois operandos e um entrada de operações anteriores, bem como, exibição dos resultados nos dois Display e avisos de *Flags* nos LEDs.

4.1. Manual de uso

Nesta seção serão fornecidas orientações sobre o manual de uso, com o objetivo de assegurar o uso adequado do sistema e ressaltar pontos importantes sobre o projeto.

4.1.1. Pinagens

Este tópico será responsável por mostrar as pinagens utilizadas no ambiente de desenvolvimento (IDE) Quartus II, declarando assim as entradas e as saídas da placa DE10-Lite, como observado na figura 13.

	Node Name	Direction	Location	I/O Bank
in	A[3]	Input	PIN_A14	7
in	A[2]	Input	PIN_A13	7
in	A[1]	Input	PIN_B12	7
in	A[0]	Input	PIN_A12	7
in	B[3]	Input	PIN_C12	7
in	B[2]	Input	PIN_D12	7
in	B[1]	Input	PIN_C11	7
in	B[0]	Input	PIN_C10	7
in	Cin	Input	PIN_B14	7
in	Selecao[2]	Input	PIN_F15	7
in	Selecao[1]	Input	PIN_A7	7
in	Selecao[0]	Input	PIN_B8	7
out	a	Output	PIN_C14	7
out	b	Output	PIN_E15	7
out	c	Output	PIN_C15	7
out	d	Output	PIN_C16	7
out	e	Output	PIN_E16	7
out	f	Output	PIN_D17	7
out	fdiv	Output	PIN_A11	7
out	flag_bout	Output	PIN_E14	7
out	flag_cout	Output	PIN_D14	7
out	flag_est	Output	PIN_C13	7
out	fzero	Output	PIN_B11	7
out	g	Output	PIN_C17	7
out	h	Output	PIN_C18	7
out	i	Output	PIN_D18	6
out	j	Output	PIN_E18	6
out	k	Output	PIN_B16	7
out	l	Output	PIN_A17	7
out	m	Output	PIN_A18	7
out	n	Output	PIN_B17	7

Figura 13. Mapeamento de pinos de entrada e saída utilizados na placa DE10-Lite. (Fonte: autoria dos autores).

4.1.2. Como utilizar os componentes

Nesse campo, serão fornecidas orientações sobre o uso adequado da ULA Aritmética implementada na placa FPGA DE10-Lite, evidenciando os componentes de entradas, como chaves e botões, e as saídas, LEDS e Displays.

De acordo com as instruções do manual, realizou-se as atribuições de funcionalidades aos componentes da placa. Dessa forma, para representar os números em binário foram utilizadas as chaves que possuíam valor lógico 0 enquanto estavam abaixadas e 1 quando estavam levantadas. Com base nesse cenário, as 10 chaves foram divididas entre os operandos de 4 bits de A e B, um Carry In de operações passadas, e uma chave seletora, contudo, foi necessário a utilização dos botões para suprir a quantidade de funcionalidades, como por exemplo, as entradas das outras seleções para escolher qual operação será realizada, vale ressaltar que, os botões possuem valores lógicos altos na falta de pressão sobre eles, e somente quando pressionados atingem valor lógico baixo, conforme ilustrado na imagem 6 é evidenciado as entradas.

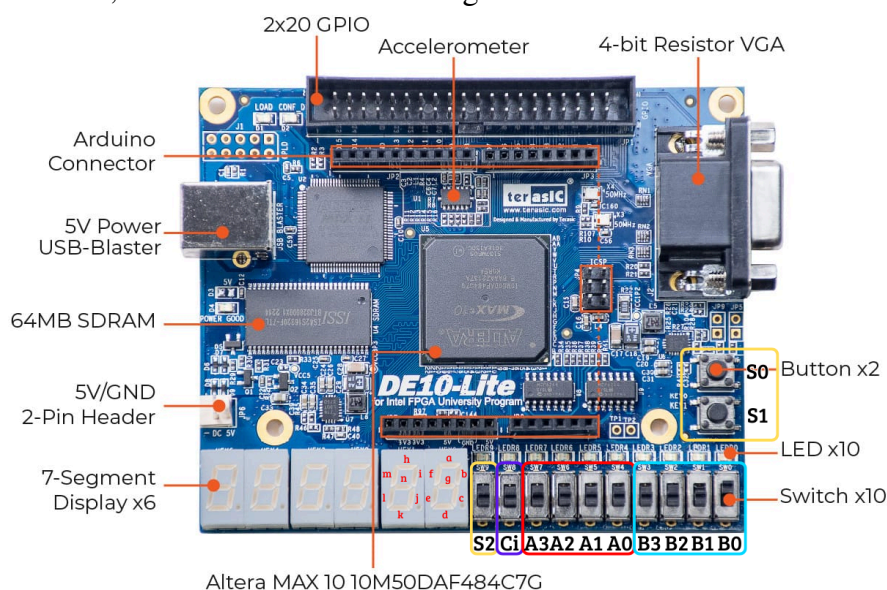


Figura 14. Placa DE10-Lite com destaque para as chaves e botões. Fonte [13].

A tabela a seguir apresenta os nomes das chaves relacionadas aos bits menos significativos (A[0] e B[0]) até os mais significativos (A[3] e B[3]) de A e B, e também o carry in.

Tabela 6. Atribuição dos operandos aos componentes físicos da placa.

B[0]	SW0
B[1]	SW1
B[2]	SW2

B[3]	SW3
A[0]	SW4
A[1]	SW5
A[2]	SW6
A[3]	SW7
Cin	SW8

4.1.3. Como escolher as operações

Neste tópico, será informado o modo de utilizar as operações aritméticas e lógicas do projeto, por meio das combinações das chaves e botões seletores, a tabela a seguir informa a correspondência com os componentes.

Tabela 7. Atribuição dos botões aos componentes físicos da placa.

S[2]	SW9
S[1]	KEY1
S[0]	KEY0

Uma vez esclarecido os atributos das peças, será informado as combinações que devem ser feitas para escolher os modos de operação.

Tabela 8. Seleção das operações.

S[2]	S[1]	S[0]	Operação
0	0	0	Soma
0	0	1	Soma
0	1	0	Subtração
0	1	1	Multiplicação
1	0	0	XOR
1	0	1	AND
1	1	0	OR
1	1	1	Divisão

4.2. Como interpretar os resultados

Os resultados do sistema são obtidos a partir das interações entre os operandos A e B, juntamente com o carry in, sendo exibidos nos Displays no intervalo de 0 a 99 em decimal. Além disso, são utilizadas *flags* de alerta para indicar situações que resultem em erro na representação.

Os Displays de 7 segmentos recebem os resultados das operações, o display HEX0 representa as unidades, enquanto o HEX1 indica as dezenas, conforme ilustrado na figura 6.



Figura 15. Representação em decimal do número 15 nos displays. Fonte: Domínio dos autores.

4.2.1. Flags

As *flags* presentes no circuito têm a função de alertar o usuário quanto à fidelidade do resultado obtido, as explicações a seguir tem o intuito de esclarecer esses possíveis erros.

1. **fzero:** Acionada quando o resultado da operação é igual a zero. Essa *flag* confirma que o valor obtido é coerente com as entradas fornecidas.
2. **fdiv:** Indicada em casos de divisão por zero. Como essa operação constitui uma indeterminação, o usuário é alertado de que a representação resultará em erro.
3. **flag_cout:** Na soma entre os operandos A e B (4 bits cada), o resultado pode ultrapassar o intervalo de 4 bits. Nessa situação, a *flag* informa que ocorreu um estouro na operação.
4. **flag_bout:** Durante a subtração, quando o valor de A é menor que o valor de B, o resultado seria negativo. Entretanto, por decisão de projeto, apenas valores inteiros e positivos são considerados. Assim, ao inserir combinações que resultam em valores negativos, essa *flag* é ativada para indicar que o resultado não será exibido corretamente.
5. **flag_est:** O intervalo de representação adotado vai de 0 a 99. Acima desse limite, o comportamento torna-se indefinido e os resultados podem aparecer de forma incorreta. Para solucionar esse problema, uma *flag* é acionada sempre que o valor ultrapassa 99, alertando o usuário.

A seguir, apresenta-se a relação das *flags* com os LEDs físicos da placa FPGA DE10-Lite, incluindo os nomes correspondentes.



Figura 16. LEDS na placa. Fonte: Domínio dos autores.

Tabela 9. Nomes das *Flags* e os seus LEDs.

fzero	LEDR9
fdiv	LEDR7
flag_cout	LEDR6
flag_bout	LEDR5
flag_est	LEDR4

4.2.2. Exemplos práticos

1. Soma de $15 + 15 = 30$ com estouro.

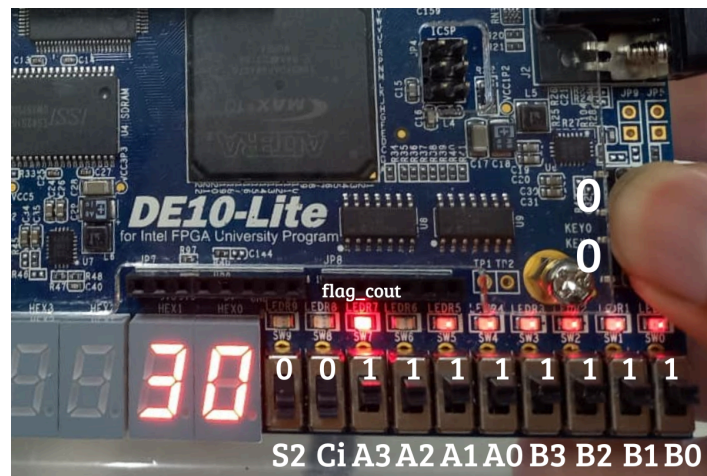


Figura 17. Operação de soma com carry out. (Fonte: autoria dos autores).

2. XOR $1 \oplus 1 = 0$ com resultado zero.

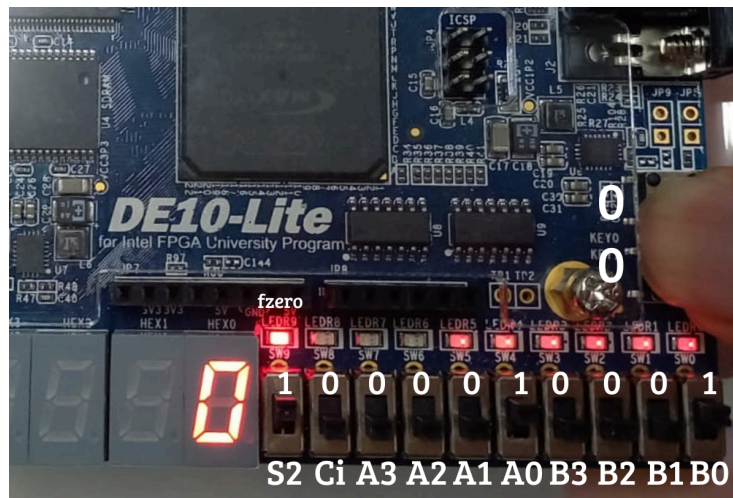


Figura 18. Operação XOR com resultado zero. Fonte: Domínio dos autores.

4.3. Discussões

Para a exibição dos resultados numéricos, o sistema foi projetado com dois displays de 7 segmentos, sendo o primeiro responsável pela representação do dígito das unidades, enquanto o segundo, pelo das dezenas. Com base nessa estrutura, elaborou-se uma Tabela Verdade para o intervalo de números entre 0 e 99, definindo para cada valor quais segmentos deveriam estar em estado ativo.

Como forma de otimizar o circuito, aplicou-se o Mapa de Karnaugh para a função lógica de cada um dos 14 segmentos. Conforme essa abordagem, que utilizou o método da Soma de Produtos (SOP), foi possível obter a expressão booleana minimizada para cada caso. Dessa forma, o processo resultou em 14 expressões simplificadas, onde cada uma é responsável por controlar o acionamento de um segmento específico para formar a representação visual correta do valor decimal.

Outra redução aplicada, foi a diminuição da complexidade e a quantidade de componentes do circuito divisor. Primeiramente, a última fileira de multiplexadores, que seria responsável por formar o resto final, foi removida. Uma vez que não há uma etapa de divisão subsequente, a seleção ou restauração do valor não é necessária. Dessa forma, o resto da divisão é obtido diretamente das saídas de diferença (Diff) do último estágio de subtração.

Uma otimização mais significativa foi aplicada aos estágios iniciais da divisão, onde o dividendo parcial é composto por 3 zeros à esquerda. Para a primeira comparação, os bits do dividendo são efetivamente 000A3. A subtração deste valor pelo divisor (B3B2B1B0) resultava em um *Borrow Out* (Bout) garantido “1” se qualquer um dos três bits mais significativos do divisor (B3, B2 ou B1) fosse “1”, pois isso tornaria o divisor inequivocamente maior.

Com base nessa lógica, a primeira fileira de subtratores pode ser substituída por uma única porta *OR* de 3 entradas, que recebe os bits B3, B2 e B1 do divisor. A saída desta porta *OR* corresponde diretamente ao *Bout* da primeira etapa. Este sinal foi utilizado para como chave de seleção para os multiplexadores da etapa seguinte e, negado pela porta *NOT*, o bit mais significativo do quociente (Q3).

Essa substituição elimina a necessidade de múltiplos subtratores e multiplexadores em cascata no estágio inicial, resultando em uma economia considerável de hardware e na potencial redução do atraso de propagação do sinal. Portanto, a implementação completa de subtratores é necessária apenas nas etapas em que os bits do dividendo parcial são significativos o suficiente para que o resultado da comparação não seja predeterminado.

5. Conclusões

O desenvolvimento da Unidade Lógica e Aritmética cumpriu os requisitos estabelecidos de implementação, permitindo a seleção entre as operações, exibição de flags e correta exibição do resultado em dois displays de 7 segmentos.

A segmentação do circuito em componentes funcionais e reutilizáveis, como somadores, multiplexadores e unidades lógicas, permitiu uma implementação estrutural didática e organizada. Cada módulo foi projetado para cumprir um requisito específico, e sua integração no módulo principal (main2) resultou em um sistema coeso e funcional.

Portanto, conclui-se que a ULA desenvolvida cumpre as funcionalidades exigidas, promovendo a resolução de problemas aritméticos básicos de forma eficiente, sendo capaz de exibir corretamente possíveis erros através das flags, e os resultados decimais nos displays.

Como forma de melhoria futura, é relevante a exibição de números acima de 99, abrangendo um display a mais do que os utilizados. Além disso, a inclusão de opções de exibição entre números decimais, binários, hexadecimais e octais.

Referências Bibliográficas

[1] CONCEIÇÃO, J. S.; SOUZA, G. S.; SILVA, C. *Estudo comparativo entre as plataformas Arduino e Raspberry Pi*.

Disponível em:
https://seer.uscs.edu.br/index.php/revista_informatica_aplicada/article/view/1665. Acessado em: 24 set. 2025.

[2] VALLE, Odilson Tadeu. *Circuitos Lógicos*. São José: Instituto Federal de Santa Catarina (IFSC), [s.d.]. Material de aula.

[3] IBM. *Quando usar FPGA ou Microcontrolador*.

Disponível em:
<https://www.ibm.com/br-pt/think/topics/fpga-vs-microco>

ntroller. Acessado em: 23 set. 2025.

- [4] WONDERFUL PCB. *O que é um FPGA e como ele funciona.*

Disponível em:
<https://www.wonderfulpcb.com/pt/blog/what-is-an-fpga-and-how-it-works/>. Acessado em: 23 set. 2025.

- [5] BDM UnB. *Implementação em FPGA de algoritmo de detecção de cantos de*

imagens para aplicações em tempo real. Disponível em:
https://bdm.unb.br/bitstream/10483/7804/1/2013_MatheusBicharadeAssumpcao.pdf. Acessado em: 23 set. 2025.

- [6] PCBASIC. *Placa FPGA: O guia completo para design, layout e fabricação.*

Disponível em:
https://www.pcbasic.com/pt/blog/fpga_board.html.
Acessado em: 23 set. 2025.

- [7] MACNICA DHW. Conheça as vantagens da tecnologia FPGA. Macnica DHW Blog, 27 abr. 2022.

Disponível em:
<https://blog.macnicadhw.com.br/treinamento-fpga/vantagens-tecnologia-fpga-intel/>. Acessado em: 23 set. 2025.

- [8] TERASIC INC. DE10-Lite User Manual. 2017.

Disponível em:
https://www.terasic.com.tw/ftpdownload_en/de10-lite_user_manual.pdf. Acessado em: 23 set. 2025.

- [9] SABER 360°. *Somadores.*

Disponível em: <https://www.saber360.com.br/somadores>.
Acessado em: 23 set. 2025.

- [10] SCRIBD. *Aula 13 - Circuitos Aritméticos - Meio Subtrator-Subtrator Completo.*

Disponível em:
<https://pt.scribd.com/presentation/543643279/Aula-13-Circuitos-Aritmeticos-Meio-Subtrator-Subtrator-Completo>.
Acessado em: 23 set. 2025.

- [11] TANKOVITS, Mihály. Logic Gates Diagram. Pinterest.

Disponível em:
<https://br.pinterest.com/pin/2181499816294535>.
Acessado em: 23 set. 2025.

- [13] TERASIC. DE10-Lite Development and Education Board.

Disponível em:
<https://www.terasic.com.tw/cgi-bin/page/archive.pl?Language=English&CategoryNo=218&No=1021&PartNo=3>. Acesso em:

22 set. 2025.

[14] **SIQUEIRA, Marcelo F.; COSTA, Umberto S.** da. *Aula 27 – Modularização:*

Parte 1. DIM0320 – Introdução à Programação. Departamento de Informática e Matemática Aplicada, Universidade Federal do Rio Grande do Norte, 2014. Disponível em: <https://www.dimap.ufrn.br/~richard/pubs/dim0320/readings/aula27.pdf>. Acesso em: 20 set. 2025.

[15] **TOCCI, Ronald J.; WIDMER, Neal S.; MOSS, Gregory L.** Sistemas Digitais:

Princípios e Aplicações. 11. ed. São Paulo: Pearson Education do Brasil, 2011.